

UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

FACULTAD DE INGENIERÍA

ARQUITECTURA DE COMPUTADORES Y ENSAMBLADORES 1 Sección A

CATEDRÁTICO: ING. OTTO RENE ESCOBAR LEIVA

TUTOR ACADÉMICO: JOSÉ MANUEL LÓPEZ LEMUS

MANUAL TÉCNICO : CASA INTELIGENTE

Grupo #3

Angel Guillermo Arreaga Barrientos	202004762
Estuardo Josué Vaquias Reyes	202201336
Pablo Roberto García Sanun	202300448
Jairo Adolfo Gomez Hernandez	201902672
Hector Daniel Ortiz Osorio	202203806
Alma Isabel Manchamé Romero	201700907

Guatemala 6 de septiembre de 2025

Introducción:

En el siguiente manual se hablara de como aplicamos las tecnologías a nuestro proyecto el cual es sobre una casa inteligente, el cerebro por así decirlo de nuestra casa será la raspberry modelo 3 con 1gb de RAM, la cual es la encargada de enviar los datos de los sensores como el de temperatura o de humedad a nuestra base de datos la cual se desarrolló en MongoDB la cual es la encargada de almacenar todos los cambios como los encendidos y apagados de las luces led, las medidas de temperatura entre otros.

Este proyecto también cuenta con una aplicación web desde la cual podemos encender o apagar las luces o el ventilador o abrir el portón, cuenta con un apartado de graficas en la cual podemos visualizar de forma gráfica cuantas veces se encendió la luz en un determinado cuarto, así mismo aparece una tabla en la cual se va registrando todo lo que está pasando junto con la fecha y la hora en la cual se activó el sensor o actuador.

Backend

- MongoService.js

este archivo nos ayuda a hablar con la base de datos en MongoDB para obtener la información de las tablas

```
JS mongoService.js X
backend > services > JS mongoService.js > ...
1  const { spawn } = require('child_process');
2
3  /**
4   * Servicio para manejar conexiones con MongoDB usando mongosh
5   * Solo operaciones de lectura para la colección illumination
6   */
7  class MongoService {
8    constructor() {
9      this.connectionString = 'mongodb+srv://estuardovaquias:qRX676MtLqktApQz@cluster0.1fjdy1g.mongodb.net/';
10     this.database = 'miBaseDeDatos';
11     this.collection = 'illumination';
12   }
13
14   /**
15    * Ejecuta un comando de mongosh
16    * @param {string} comando - Comando JavaScript para ejecutar en mongosh
17    * @returns {Promise<string>} - Resultado del comando
18    */
19   ejecutarMongosh(comando) {
20     return new Promise((resolve, reject) => {
21       const mongosh = spawn('mongosh', [
22         this.connectionString,
23         '--eval', comando,
24         '--quiet'
25       ]);
26
27       let resultado = '';
28       let error = '';
29
30       mongosh.stdout.on('data', (data) => {
31         resultado += data.toString();
32       });
33     });
34   }
35 }
```

Esta función es la base de todo. Se encarga de ejecutar comandos directamente en MongoDB usando la herramienta mongosh. Todas las demás funciones la usan para comunicarse con la base de datos.

```
ejecutarMongosh(comando) {
  return new Promise((resolve, reject) => {
    const mongosh = spawn('mongosh', [
      this.connectionString,
      '--eval', comando,
      '--quiet'
    ]);

    let resultado = '';
    let error = '';

    mongosh.stdout.on('data', (data) => {
      resultado += data.toString();
    });

    mongosh.stderr.on('data', (data) => {
      error += data.toString();
    });

    mongosh.on('close', (code) => {
      if (code === 0) {
        resolve(resultado.trim());
      } else {
        reject(new Error(`Error en mongosh (código ${code}): ${error}`));
      }
    });
  });
}
```

Obtiene todos los registros almacenados en la colección de iluminación. Sirve para mostrar el historial completo de eventos de luces en el dashboard

```
async getAllIllumination() {
    return await this.getAllFromCollection('iluminacion');
}

/**
 * Busca un registro de iluminación por nombre
 * @param {string} nombre - Nombre a buscar
 * @returns {Promise<Object|null>} - Registro encontrado o null
 */
async getIlluminationByName(nombre) {
    try {
        if (!nombre || typeof nombre !== 'string') {
            throw new Error('El nombre es requerido y debe ser una cadena');
        }

        const comando = `
            use('${this.database}');
            const usuario = db.iluminacion.findOne({ nombre: "${nombre}" });
            print(JSON.stringify(usuario));
        `;

        const resultado = await this.ejecutarMongosh(comando);

        if (!resultado || resultado === 'null') {
            console.log(`🔍 No se encontró registro con nombre: ${nombre}`);
            return null;
        }
    }
}
```

Busca un registro específico en la base de datos usando el nombre como filtro. útil para encontrar información de una habitación o luz en particular

```
*/
async getCollectionStats(collectionName) {
    try {
        if (!this.isValidCollection(collectionName)) {
            throw new Error(`Colección '${collectionName}' no válida`);
        }

        const comando = `
            use('${this.database}');
            const count = db.${collectionName}.countDocuments();
            const stats = db.${collectionName}.stats();
            print(JSON.stringify({
                collection: "${collectionName}",
                totalDocuments: count,
                collectionSize: stats.size,
                avgDocumentSize: stats.avgObjSize,
                indexes: stats.nindexes
            }));
        `;

        const resultado = await this.ejecutarMongosh(comando);
        const stats = JSON.parse(resultado);

        console.log(`✅ Estadísticas de ${collectionName} obtenidas`);
    }
}
```

- mqttService.js

Este archivo es un servicio que maneja toda la comunicación con el broker MQTT (HiveMQ Cloud). Su función es conectar nuestro backend con el sistema de mensajería MQTT para enviar y recibir mensajes en tiempo real con la Raspberry Pi

Obtiene la fecha y hora actual en formato local (GMT-6). Se usa para timestamp en mensajes y alertas

```
function getCurrentDateTime() {
  const now = new Date();
  const gmtMinus6 = new Date(now.getTime() - (6 * 60 * 60 * 1000));

  const day = String(gmtMinus6.getUTCDate()).padStart(2, '0');
  const month = String(gmtMinus6.getUTCMonth() + 1).padStart(2, '0');
  const year = gmtMinus6.getUTCFullYear();
  const fecha = `${day}-${month}-${year}`;

  const hours = String(gmtMinus6.getUTCHours()).padStart(2, '0');
  const minutes = String(gmtMinus6.getUTCMinutes()).padStart(2, '0');
  const hora = `${hours}:${minutes}`;

  const timestamp = `${fecha} ${hora} GMT-6`;

  return { fecha, hora, timestamp };
}
```

Inicializa la conexión con el broker MQTT y se suscribe al tópico de alertas. Es lo primero que se debe llamar para usar el servicio

```
init() {
  return new Promise((resolve, reject) => {
    console.log('🔌 Iniciando conexión MQTT...');

    this.client = mqtt.connect(this.config.url, {
      username: this.config.username,
      password: this.config.password,
      clientId: this.config.clientId,
      keepalive: this.config.keepalive,
      clean: this.config.clean,
      reconnectPeriod: this.config.reconnectPeriod,
      connectTimeout: this.config.connectTimeout
    });

    this.client.on("connect", () => {
      console.log("✅ Conectado exitosamente a HiveMQ Cloud");
      this.isConnectedFlag = true;

      // Suscribirse a alertas
      this.client.subscribe("/alerts", { qos: 1 }, (err) => {
        if (err) {

```

Procesa los mensajes que llegan de MQTT. especialmente los del tópic /alerts, guardándolos en una lista

```
//
handleMessage(topic, message) {
  const messageStr = message.toString();
  console.log(`📄 Mensaje recibido en ${topic}:`, messageStr);

  if (topic === "/alerts") {
    const dateTime = getCurrentDateTime();

    const newAlert = {
      id: Date.now(),
      message: messageStr,
      fecha: dateTime.fecha,
      hora: dateTime.hora,
      timestamp: dateTime.timestamp,
      topic: topic
    };

    // Mantener solo los últimos 10 alerts
    this.alerts = [newAlert, ...this.alerts.slice(0, 9)];
    console.log(`🔴 Nueva alerta agregada. Total: ${this.alerts.length}`);
  }
}
```

- server.js

Este es el archivo principal del backend que crea el servidor web (con Express) y define todas las rutas de la API. Es el centro de control que recibe peticiones del frontend, se comunica con la base de datos (MongoService) y envía comandos a los dispositivos (MQTTService)

Muestra las colecciones disponibles en la base de datos, que seria donde se almacenan nuestros datos

```
// Obtener lista de colecciones disponibles
app.get('/api/collections', (req, res) => {
  try {
    const collections = mongoService.getAvailableCollections();
    res.json({
      success: true,
      data: collections,
      message: 'Colecciones disponibles'
    });
  } catch (error) {
    res.status(500).json({
      success: false,
      error: error.message
    });
  }
});

// Obtener estadísticas de todas las colecciones
```

Obtiene todos los registros de iluminación de la base de datos, nos sirve para poder crear las estadísticas en el frontend, y la siguiente función hace que obtengamos el nombre específico de la habitación en la cual se encendió el led.

```
// Obtener todos los registros de iluminación
app.get('/api/iluminacion', async (req, res) => {
  try {
    const illumination = await mongoService.getAllIllumination();
    res.json({
      success: true,
      data: illumination
    });
  } catch (error) {
    console.error('Error al obtener iluminación:', error);
    res.status(500).json({
      success: false,
      error: error.message
    });
  }
});
```

```
// Buscar registro de iluminación por nombre
app.get('/api/iluminacion/:nombre', async (req, res) => {
  try {
    const { nombre } = req.params;
    const illumination = await mongoService.getIlluminationByName(nombre);

    if (illumination) {
      res.json({
        success: true,
        data: illumination
      });
    } else {
      res.status(404).json({
        success: false,
        message: 'Registro de iluminación no encontrado'
      });
    }
  } catch (error) {
    console.error('Error al buscar iluminación:', error);
    res.status(500).json({
      success: false,
      error: error.message
    });
  }
});
```

Obtiene datos del sensor de humedad, que están almacenados en la base de datos

```

// Obtener todos los registros de humedad del suelo
app.get('/api/humedad-suelo', async (req, res) => {
  try {
    const { limit } = req.query;
    const data = limit ?
      await mongoService.getFromCollectionWithLimit('humedad_suelo', parseInt(limit)) :
      await mongoService.getAllFromCollection('humedad_suelo');

    res.json({
      success: true,
      data: data,
      count: data.length
    });
  } catch (error) {
    console.error('Error al obtener humedad del suelo:', error);
    res.status(500).json({
      success: false,
      error: error.message
    });
  }
});

```

Obtiene registros del sensor de movimiento

```

// Obtener todos los registros de movimiento
app.get('/api/movimiento', async (req, res) => {
  try {
    const { limit } = req.query;
    const data = limit ?
      await mongoService.getFromCollectionWithLimit('movimiento', parseInt(limit)) :
      await mongoService.getAllFromCollection('movimiento');

    res.json({
      success: true,
      data: data,
      count: data.length
    });
  } catch (error) {
    console.error('Error al obtener datos de movimiento:', error);
    res.status(500).json({
      success: false,
      error: error.message
    });
  }
});

```


Muestra eventos de riego

```
// Obtener todos los registros de eventos de riego
app.get('/api/riego-eventos', async (req, res) => {
  try {
    const { limit } = req.query;
    const data = limit ?
      await mongoService.getFromCollectionWithLimit('riego_eventos', parseInt(limit)) :
      await mongoService.getAllFromCollection('riego_eventos');

    res.json({
      success: true,
      data: data,
      count: data.length
    });
  } catch (error) {
    console.error('Error al obtener eventos de riego:', error);
    res.status(500).json({
      success: false,
      error: error.message
    });
  }
});
```

Para obtener los datos de la base sobre la temperatura

```
// Obtener todos los registros de temperatura
app.get('/api/temperatura', async (req, res) => {
  try {
    const { limit } = req.query;
    const data = limit ?
      await mongoService.getFromCollectionWithLimit('temperatura', parseInt(limit)) :
      await mongoService.getAllFromCollection('temperatura');

    res.json({
      success: true,
      data: data,
      count: data.length
    });
  } catch (error) {
    console.error('Error al obtener datos de temperatura:', error);
    res.status(500).json({
      success: false,
      error: error.message
    });
  }
});
```

Muestra el estado del servidor y todas las conexiones

```
// Status del servidor y conexiones
app.get('/api/status', (req, res) => {
  const dateTime = getCurrentDateTime();

  res.json({
    server: 'running',
    mqtt_connected: mqttService.isConnected(),
    mongodb_available: true,
    fecha: dateTime.fecha,
    hora: dateTime.hora,
    timestamp: dateTime.timestamp
  });
});
```

Controla el encendido y apagado de la habitación 1 o 2 o la sala, también guarda datos como la fecha y la hora

```
// Controlar habitaciones normales (encender/apagar)
app.post('/api/rooms/:room/toggle', async (req, res) => {
  const { room } = req.params;
  const { state } = req.body; // true = encender, false = apagar

  // Validar habitación
  if (!['sala', 'cuarto1', 'cuarto2'].includes(room)) {
    return res.status(400).json({
      success: false,
      error: 'Habitación no válida'
    });
  }

  try {
    const dateTime = getCurrentDateTime();

    const message = {
      device: "led_room",
      room,
      action: state ? "on" : "off",
      fecha: dateTime.fecha,
      hora: dateTime.hora,
      timestamp: dateTime.timestamp
    };
  }
});
```

Maneja el color del led RGB y controla el encendido o apagado del led

```
// Controlar habitación RGB
app.post('/api/rgb/toggle', async (req, res) => {
  const { state } = req.body;

  try {
    const rgbComponents = mqttService.hexToRgb(roomStates.rgb_room.color);
    const dateTime = getCurrentDateTime();

    const message = {
      device: "led_rgb",
      action: state ? "on" : "off",
      r: rgbComponents.r,
      g: rgbComponents.g,
      b: rgbComponents.b,
      fecha: dateTime.fecha,
      hora: dateTime.hora,
      timestamp: dateTime.timestamp
    };

    await mqttService.publish("/iluminacion", message);

    roomStates.rgb_room.isOn = state;

    res.json({
      success: true,
      data: {
        room: 'rgb_room',
        state,
        color: roomStates.rgb_room.color,
        message: 'Habitación RGB ${state ? 'encendida' : 'apagada'}',
        fecha: dateTime.fecha,

```

Y para enviar el color que se eligió en el frontend

```
// Cambiar color RGB
app.post('/api/rgb/color', async (req, res) => {
  const { color } = req.body;

  if (!color || !color.match(/^#[0-9A-F]{6}$/i)) {
    return res.status(400).json({
      success: false,
      error: 'Color hex no válido'
    });
  }

  try {
    const r = parseInt(color.substr(1, 2), 16);
    const g = parseInt(color.substr(3, 2), 16);
    const b = parseInt(color.substr(5, 2), 16);
    const dateTime = getCurrentDateTime();

    const message = {
      device: "led_rgb",
      action: "color",
      r, g, b,
      fecha: dateTime.fecha,
      hora: dateTime.hora,
      timestamp: dateTime.timestamp
    };

    await mqttService.publish("/iluminacion", message);

    roomStates.rgb_room.color = color;
  } catch (error) {
    console.log(error);
  }
});
```

Para abrir y cerrar el portón desde el frontend

```
// Controlar portón
app.post('/api/entrance/:action', async (req, res) => {
  const { action } = req.params;

  if (!['open', 'close'].includes(action)) {
    return res.status(400).json({
      success: false,
      error: 'Acción no válida. Use "open" o "close"'
    });
  }

  try {
    const dateTime = getCurrentDateTime();

    const message = {
      device: "entrance",
      action,
      fecha: dateTime.fecha,
      hora: dateTime.hora,
      timestamp: dateTime.timestamp
    };

    await mqttService.publish("/entrance", message);
  }
});
```

Las funciones para inicializar el servicio de mqtt y la conexión de la base de datos.

```
async function initializeServices() {
  try {
    console.log('🔧 Iniciando servicios...');

    // Inicializar MQTT
    await mqttService.init();
    console.log('✅ MQTT inicializado');

    // Probar conexión MongoDB
    const mongoConnected = await mongoService.testConnection();
    console.log(mongoConnected ? '✅ MongoDB conectado' : '⚠️ MongoDB no disponible');

    console.log('✅ Todos los servicios inicializados correctamente');
  } catch (error) {
    console.error('❌ Error al inicializar servicios:', error);
  }
}

// Inicializar servicios y servidor
initializeServices();

app.listen(port, () => {
  const dateTime = getCurrentDateTime();
  console.log(`🚀 Servidor corriendo en puerto ${port}`);
  console.log(`📡 API disponible en http://localhost:${port}/api`);
  console.log(`📖 Documentación en http://localhost:${port}/api`);
  console.log(`🕒 Iniciado el ${dateTime.timestamp}`);
});

module.exports = app;
```

Frontend

- Login.js

```
const Login = () => {  
  const [credentials, setCredentials] = useState({  
    username: "",  
    password: "",  
  });  
  const [error, setError] = useState("");  
  const [isLoading, setIsLoading] = useState(false);  
  const navigate = useNavigate();
```

para guardar el usuario y la contraseña y por si algún dato está mal, isLoading: indica si se está procesando el login y el navigate: permite cambiar de ruta (página) tras login exitoso

```
    setTimeout(() => {  
      if (credentials.username === "123" && credentials.password === "123") {  
        // Guardar estado de autenticación (solo para esta sesión)  
        sessionStorage.setItem("isAuthenticated", "true");  
        // Navegar al dashboard  
        navigate('/dashboard', { replace: true });  
      } else {  
        setError("Credenciales incorrectas. Por favor, intente nuevamente.");  
      }  
      setIsLoading(false);  
    }, 1000);  
  }  
};
```

se comparan las credenciales para que el usuario pueda iniciar sesión y si en caso no es que vuelva a ingresar la contraseña

```
    return (  
      <div className="min-h-screen bg-gradient-to-br from-blue-50 via-white to-purple-50 flex items-center justify-center">  
        <div className="bg-white rounded-xl shadow-2xl p-8 w-full max-w-md">  
          <div className="text-center mb-8">  
            <h1 className="text-3xl font-bold text-gray-800 mb-2">  
              Dashboard IoT  
            </h1>  
            <p className="text-gray-600">Proyecto 1 - Grupo 3</p>  
          </div>  
          <form onSubmit={handleSubmit} className="space-y-6">  
            {error && (  
              <div className="bg-red-50 border border-red-200 rounded-lg p-4 flex items-center gap-3">  
                <AlertCircle className="w-5 h-5 text-red-500 flex-shrink-0" />  
                <p className="text-red-700 text-sm">{error}</p>  
              </div>  
            )}  
            <div className="space-y-2">  
              <label>  
                htmlFor="username"  
                className="block text-sm font-medium text-gray-700">  
                Usuario  
              </label>  
              <div className="relative">  
                <div className="absolute inset-y-0 left-0 pl-3 flex items-center pointer-events-none">  
                  <User className="h-5 w-5 text-gray-400" />  
                <input type="text" className="block w-full pl-10 text-gray-900" />  
              </div>  
            </div>  
            <div className="space-y-2">  
              <label>  
                htmlFor="password"  
                className="block text-sm font-medium text-gray-700">  
                Contraseña  
              </label>  
              <div className="relative">  
                <div className="absolute inset-y-0 left-0 pl-3 flex items-center pointer-events-none">  
                  <Eye className="h-5 w-5 text-gray-400" />  
                <input type="password" className="block w-full pl-10 text-gray-900" />  
              </div>  
            </div>  
            <button type="submit" className="w-full text-white bg-blue-600 hover:bg-blue-700 font-medium rounded-lg text-sm px-5 py-3">  
              Iniciar Sesión  
            </button>  
          </form>  
        </div>  
      </div>
```

y el formulario del login en general y la estructura

- Dashboard.js

```
// Función para hacer peticiones a la API
const apiRequest = async (endpoint, options = {}) => {
  try {
    const response = await fetch(`${API_BASE_URL}${endpoint}`, {
      headers: {
        'Content-Type': 'application/json',
        ...options.headers
      },
      ...options
    });
    if (!response.ok) {
      throw new Error(`HTTP error! status: ${response.status}`);
    }
    return await response.json();
  } catch (error) {
    console.error(`Error en API ${endpoint}:`, error);
    throw error;
  }
};
```

Propósito: Función genérica para hacer peticiones HTTP al backend

Uso: Centraliza todas las llamadas a la API con manejo de errores

```
// Verificar estado del backend y cargar datos iniciales
useEffect(() => {
  const checkBackendStatus = async () => {
    try {
      const status = await apiRequest('/status');
      setBackendStatus(true);
      setIsConnected(status.mqtt_connected);
      setMongoStatus(status.mongodb_connected || status.mongodb_available);
    } catch (error) {
      setBackendStatus(false);
      setIsConnected(false);
      setMongoStatus(false);
      console.error('Backend no disponible:', error);
    }
  };
});
```

Propósito: Cargar datos iniciales y configurar intervalos de actualización

Uso: Verifica estado del backend, carga estados de habitaciones y alertas

```

// Control de habitaciones normales
const toggleRoom = async (room, state) => {
  if (!backendStatus) {
    alert('Backend no disponible');
    return;
  }

  setLoading(true);
  try {
    const response = await apiRequest('/mqtt/rooms/${room}/toggle', { // ✅ Agregada /
      method: 'POST',
      body: JSON.stringify({ state })
    });

    if (response.success) {
      setRoomStates(prev => ({
        ...prev,
        [room]: state
      }));
    } else {
      alert('Error: ' + response.error);
    }
  } catch (error) {
    alert('Error al controlar la habitación: ' + error.message);
  } finally {
    setLoading(false);
  }
}

```

Propósito: Encender/apagar habitaciones normales (sala, cuarto1, cuarto2)

Uso: Control de luces básicas

```

// Control de habitación RGB - toggle
const toggleRgbRoom = async (state) => {
  if (!backendStatus) {
    alert('Backend no disponible');
    return;
  }

  setLoading(true);
  try {
    const response = await apiRequest('/mqtt/rgb/toggle', { // ✅ Agregada /
      method: 'POST',
      body: JSON.stringify({ state })
    });

    if (response.success) {
      setRoomStates(prev => ({
        ...prev,
        rgb_room: { ...prev.rgb_room, isOn: state }
      }));
    } else {
      alert('Error: ' + response.error);
    }
  } catch (error) {
    alert('Error al controlar la habitación RGB: ' + error.message);
  } finally {
    setLoading(false);
  }
}

```

Propósito: Encender/apagar la habitación RGB

Uso: Control del estado on/off de luces RGB

```
// Control de color RGB
const sendRgbColor = async (color) => {
  if (!backendStatus) {
    alert('Backend no disponible');
    return;
  }

  try {
    const response = await apiRequest('/mqtt/rgb/color', { // ✅ Agregada /
      method: 'POST',
      body: JSON.stringify({ color })
    });

    if (response.success) {
      setRoomStates(prev => ({
        ...prev,
        rgb_room: { ...prev.rgb_room, color }
      }));
      setSelectedColor(color);
    } else {
      alert('Error: ' + response.error);
    }
  } catch (error) {
    console.error('Error al cambiar color:', error);
  }
};
```

Propósito: Cambiar el color de las luces RGB

Uso: Envía el color seleccionado al backend

```
// Control del porton
const controlEntrance = async (action) => {
  if (!backendStatus) {
    alert('Backend no disponible');
    return;
  }

  setLoading(true);
  try {
    const response = await apiRequest(`/mqtt/entrance/${action}`, { // ✅ Agregada /
      method: 'POST'
    });

    if (response.success) {
      setEntranceState(action === 'open' ? 'opening' : 'closing');

      setTimeout(() => {
        setEntranceState(action === 'open' ? 'open' : 'closed');
      }, 2000);
    } else {
      alert('Error: ' + response.error);
    }
  } catch (error) {
    alert('Error al controlar el portón: ' + error.message);
  } finally {
    setLoading(false);
  }
};
```

Propósito: Abrir/cerrar el portón principal

Uso: Control del portón con animaciones de estado


```

// Control del ventilador
const toggleVentilador = async (state) => {
  if (!backendStatus) {
    alert('Backend no disponible');
    return;
  }

  setLoading(true);
  try {
    const response = await apiRequest('/mqtt/ventilador/toggle', { // ✅ Agregada /
      method: 'POST',
      body: JSON.stringify({ state })
    });

    if (response.success) {
      setRoomStates(prev => ({
        ...prev,
        ventilador: state
      }));
    } else {
      alert('Error: ' + response.error);
    }
  } catch (error) {
    alert('Error al controlar el ventilador: ' + error.message);
  } finally {
    setLoading(false);
  }
}

```

Propósito: Encender/apagar el ventilador

Uso: Control del dispositivo de ventilación

```

// Control de la bomba de agua
const toggleBombaAgua = async (state) => {
  if (!backendStatus) {
    alert('Backend no disponible');
    return;
  }

  setLoading(true);
  try {
    const response = await apiRequest('/mqtt/bomba-agua/toggle', { // ✅ Agregada /
      method: 'POST',
      body: JSON.stringify({ state })
    });

    if (response.success) {
      setRoomStates(prev => ({
        ...prev,
        bomba_agua: state
      }));
    } else {
      alert('Error: ' + response.error);
    }
  } catch (error) {
    alert('Error al controlar la bomba de agua: ' + error.message);
  } finally {
    setLoading(false);
  }
}

```

Propósito: Activar/desactivar la bomba de agua

Uso: Control del sistema de agua

```
// Control de la alarma
const toggleAlarma = async (state) => {
  if (!backendStatus) {
    alert('Backend no disponible');
    return;
  }

  setLoading(true);
  try {
    const response = await apiRequest('/mqtt/alarma/toggle', {
      method: 'POST',
      body: JSON.stringify({ state })
    });

    if (response.success) {
      setRoomStates(prev => ({
        ...prev,
        alarma: state
      }));
    } else {
      alert('Error: ' + response.error);
    }
  } catch (error) {
    alert('Error al controlar la alarma: ' + error.message);
  } finally {
    setLoading(false);
  }
}
```

Propósito: Activar/desactivar el sistema de alarma

Uso: Control de seguridad

Statistics.js

```
// Función para hacer peticiones a la API
const apiRequest = async (endpoint, options = {}) => {
  try {
    const response = await fetch(`${API_BASE_URL}${endpoint}`, {
      headers: {
        'Content-Type': 'application/json',
        ...options.headers
      },
      ...options
    });

    if (!response.ok) {
      throw new Error(`HTTP error! status: ${response.status}`);
    }

    return await response.json();
  } catch (error) {
    console.error(`Error en API ${endpoint}:`, error);
    throw error;
  }
};
```

Función genérica para realizar peticiones HTTP al backend. Maneja errores y configuraciones de headers.

```
const loadData = async () => {
  setIsLoading(true);
  try {
    // Verificar backend
    await apiRequest('/status');
    setBackendStatus(true);

    // Cargar estados de habitaciones sin depender de fecha)
    const roomsResponse = await apiRequest('/mqtt/rooms');
    if (roomsResponse.success) {
      setRoomStates(roomsResponse.data);
    }

    // Cargar alertas (no dependen de fecha)
    const alertsResponse = await apiRequest('/mqtt/alerts');
    if (alertsResponse.success) {
      setAlerts(alertsResponse.data);
    }

    // Cargar TODOS los datos sin filtros
    const temperatureResponse = await apiRequest('/temperatura');
    if (temperatureResponse.success) {
      setTemperatureRawData(temperatureResponse.data);
    }

    const soilMoistureResponse = await apiRequest('/humedad-suelo');
    if (soilMoistureResponse.success) {
      setSoilMoistureRawData(soilMoistureResponse.data);
    }
  }
}
```

Carga todos

los datos iniciales del sistema: estados de habitaciones, alertas, temperatura, humedad, movimiento e iluminación.

```
// Funcion para obtener la fecha correcta de un item
const getItemTimestamp = (item) => {
  // Para datos de iluminación, usar timestampMsg o receivedAt
  if (item.timestampMsg) return item.timestampMsg;
  if (item.receivedAt) return item.receivedAt;
  // Para otros sensores
  return item.payload?.timestamp || item.fechaHora;
};
```

Obtiene la fecha/hora de un item, manejando diferentes formatos de timestamp (timestampMsg, receivedAt, payload.timestamp, fechaHora).

```

// Funcion para filtrar datos por fecha y hora
const filterDataByDateTime = (data, startDate, endDate, startTime, endTime) => {
  const startDateTime = new Date(`${startDate}T${startTime}`);
  const endDateTime = new Date(`${endDate}T${endTime}`);

  return data.filter(item => {
    const itemTimestamp = getItemTimestamp(item);
    if (!itemTimestamp) return false;

    let itemDate;
    // Manejar diferentes formatos de fecha
    if (itemTimestamp.includes('GMT')) {
      // Formato: "03-09-2025 17:53 GMT-6"
      const dateTimeStr = itemTimestamp.replace(' GMT-6', '');
      const [datePart, timePart] = dateTimeStr.split(' ');
      const [day, month, year] = datePart.split('-');
      itemDate = new Date(`${year}-${month}-${day}T${timePart}`);
    } else {
      itemDate = new Date(itemTimestamp);
    }

    return itemDate >= startDateTime && itemDate <= endDateTime;
  });
};

```

Filtra datos por rango de fecha y hora especificado, convirtiendo diferentes formatos de fecha a objetos Date.

```

// Procesar datos de temperatura FILTRADOS
const processFilteredTemperatureData = (data) => {
  const tempData = data
    .filter(item => item.tempC !== undefined)
    .map(item => ({
      x: new Date(getItemTimestamp(item)),
      y: item.tempC
    }));

  setTemperatureData(tempData);

  // wxtraer datos de humedad si estn disponibles
  const humidityData = data
    .filter(item => item.humedad !== undefined)
    .map(item => ({
      x: new Date(getItemTimestamp(item)),
      y: item.humedad
    }));

  setHumidityData(humidityData);
};

```

Procesa datos de temperatura filtrados para gráficas, extrayendo valores de temperatura y humedad ambiental.

```
// Procesar datos de humedad del suelo FILTRADOS
const processFilteredSoilMoistureData = (data) => {
  const moistureCounts = {
    wet: 0,
    dry: 0
  };

  data.forEach(item => {
    if (item.state === 'wet') {
      moistureCounts.wet++;
    } else if (item.state === 'dry') {
      moistureCounts.dry++;
    }
  });

  setSoilMoistureData(moistureCounts);
};
```

Cuenta eventos de suelo húmedo y seco para gráficas de humedad del suelo.

```
// Procesar datos de movimiento FILTRADOS
const processFilteredMovementData = (data) => {
  const movementCount = data.filter(item =>
    item.accion && item.accion.includes('MOVIMIENTO ON'))
    .length;

  setMovementData(movementCount);
};
```

Cuenta detecciones de movimiento basado en mensajes "MOVIMIENTO ON".

```
// Procesar datos de iluminación FILTRADOS para extraer información de actuadores
const processFilteredIlluminationData = (data) => {
  const actuatorCounts = {};
  data.forEach(event => {
    if (event.device) {
      const deviceType = event.device;
      if (!actuatorCounts[deviceType]) {
        actuatorCounts[deviceType] = 0;
      }
      actuatorCounts[deviceType]++;
    }
  });
  setActuatorData(actuatorCounts);
};
```

Extrae y cuenta activaciones de actuadores (dispositivos) desde datos de iluminación.

```
// Función para aplicar filtros a TANTO gráficas COMO tablas
const applyFilters = () => {
  const { start, end, startTime, endTime } = dateRange;

  // Filtrar datos de temperatura
  const filteredTempData = filterDataByDateTime(temperatureRawData, start, end, startTime, endTime);
  setFilteredTemperatureData(filteredTempData);
  processFilteredTemperatureData(filteredTempData);

  // Filtrar datos de humedad del suelo
  const filteredSoilData = filterDataByDateTime(soilMoistureRawData, start, end, startTime, endTime);
  processFilteredSoilMoistureData(filteredSoilData);

  // Filtrar datos de movimiento
  const filteredMovementData = filterDataByDateTime(movementRawData, start, end, startTime, endTime);
  processFilteredMovementData(filteredMovementData);

  // Filtrar datos de iluminación/actuadores
  const filteredRawData = filterDataByDateTime(rawData, start, end, startTime, endTime);
  processFilteredIlluminationData(filteredRawData);

  // Combinar todos los datos y filtrar para la tabla general
  const allData = [
    ...filteredRawData.map(item => ({...item, deviceType: 'iluminacion'})),
    ...filteredTempData.map(item => ({...item, device: 'temperatura'})),
    ...filteredSoilData.map(item => ({...item, device: 'humedad_suelo'})),
    ...filteredMovementData.map(item => ({...item, device: 'movimiento'}))
  ];
};
```

Aplica todos los filtros configurados a los datos crudos y actualiza tanto gráficas como tablas.

```
// Función para manejar la aplicación de filtros
const handleApplyFilters = () => {
  setIsLoading(true);
  setTimeout(() => {
    applyFilters();
    setIsLoading(false);
  }, 500);
};
```

Maneja la aplicación de filtros con estado de carga visual para el usuario.

```
// Función para formatear el valor a mostrar en las tablas
const formatValue = (item) => {
  if (item.tempC !== undefined) {
    return `${item.tempC} °C` + (item.humedad !== undefined ? `, ${item.humedad}% humedad` : '');
  } else if (item.state !== undefined) {
    return item.state === 'wet' ? 'Húmedo' : 'Seco';
  } else if (item.action !== undefined) {
    return `${item.action}` + (item.area ? ` (${item.area})` : '');
  } else if (item.action !== undefined) {
    let actionText = `${item.action}`;
    if (item.room) actionText += ` (${item.room})`;
    // Para datos RGB, agregar información de color
    if (item.device === 'led_rgb' && item.r !== undefined) {
      actionText += ` - RGB(${item.r}, ${item.g}, ${item.b})`;
    }
    return actionText;
  } else if (item.value !== undefined) {
    return `${item.value}`;
  }
  return 'N/A';
};
```

Formatea valores para mostrar en tablas, convirtiendo datos crudos a texto legible según el tipo de sensor.

App.py

```
def ejecutar_mongosh(comando: str) -> str:
    try:
        res = subprocess.run(
            ["mongosh", MONGO_URI, "--eval", comando, "--quiet"],
            capture_output=True, text=True, check=True
        )
        return res.stdout.strip()
    except subprocess.CalledProcessError as e:
        raise Exception(e.stderr.strip() or "Error ejecutando mongosh")

def guardar_evento(doc: dict, coleccion: str = COLL_NAME):
    doc_json = json.dumps(doc, ensure_ascii=False).replace("\\", "\\\\").replace("'", "\\'")
    comando = f"""
        use('{DB_NAME}');
        const doc = JSON.parse('{doc_json}');
        const r = db.{coleccion}.insertOne(doc);
        print(JSON.stringify({{ insertedId: r.insertedId }}));
    """
    try:
        out = ejecutar_mongosh(comando)
        if out:
            print(f" Guardado en MongoDB ({coleccion}) -> {out}")
    except Exception as e:
        print(f" No se pudo guardar en MongoDB: {e}")

def utcnow_iso():
```

Ejecuta comandos en MongoDB mediante subprocessos. Se usa para insertar documentos en la base de datos.

```
def lcd_write_lines(line1: str, line2: str | None = None):
    if not lcd: return
    with _lcd_lock:
        try:
            lcd.clear()
            lcd.write_string(line1[:16])
            if line2 is not None:
                lcd.crlf()
                lcd.write_string(line2[:16])
        except Exception as e:
            print(f" LCD error: {e}")
```

Controla la pantalla LCD mostrando mensajes en dos líneas. Maneja errores de hardware.


```
def lcd_show_temp(temp_c: float, hum: float):
    text = f"T:{temp_c:4.1f}C H:{hum:4.1f}%"
    global _last_temp_text
    _last_temp_text = text
    if not lcd: return
    with _lcd_lock:
        try:
            lcd.home()
            lcd.write_string(text[:16].ljust(16))
        except Exception as e:
            print(f" LCD error: {e}")
```

Muestra temperatura y humedad en la primera línea del LCD manteniendo formato consistente.

```
def lcd_show_message(line2: str, duration_sec: int = 4):
    if not lcd: return
    global _lcd_msg_timer
    with _lcd_lock:
        try:
            lcd.home()
            lcd.write_string(_last_temp_text[:16].ljust(16))
            lcd.crlf()
            lcd.write_string(line2[:16].ljust(16))
        except Exception as e:
            print(f" LCD error: {e}")
    if _lcd_msg_timer:
        try: _lcd_msg_timer.cancel()
        except Exception: pass
    _lcd_msg_timer = threading.Timer(duration_sec, _lcd_clear_line2)
    _lcd_msg_timer.daemon = True
    _lcd_msg_timer.start()
```

Muestra mensajes temporales en la segunda línea del LCD con temporizador para autolimpiar.

```
# ===== RGB NORMAL =====
ACTIVE_HIGH = True
RGB = {
    "r": PWMLED(5, active_high=ACTIVE_HIGH), # Físico 29
    "g": PWMLED(6, active_high=ACTIVE_HIGH), # Físico 31
    "b": PWMLED(13, active_high=ACTIVE_HIGH), # Físico 33
}
def rgb_off():
    RGB["r"].off(); RGB["g"].off(); RGB["b"].off()
def set_rgb_values(r: float, g: float, b: float):
    RGB["r"].value, RGB["g"].value, RGB["b"].value = r, g, b
def off_rgb():
    rgb_off()

# ===== RGB ALARMA (ROJO/AZUL) =====
```

Controla el led RGB

```
# ===== RGB ALARMA (ROJO/AZUL) =====
ALARM_ACTIVE_HIGH = ACTIVE_HIGH
ALARM_RGB = {
    "r": PWMLED(16, active_high=ALARM_ACTIVE_HIGH), # GPIO16 (Físico 36)
    "g": PWMLED(20, active_high=ALARM_ACTIVE_HIGH), # GPIO20 (Físico 38)
    "b": PWMLED(21, active_high=ALARM_ACTIVE_HIGH), # GPIO21 (Físico 40)
}
def alarm_rgb_off():
    ALARM_RGB["r"].off(); ALARM_RGB["g"].off(); ALARM_RGB["b"].off()
_alarm_rgb_stop = threading.Event()
def _alarm_rgb_worker():
    toggle = False
    while not _alarm_rgb_stop.is_set():
        if toggle:
            # Alarma Activada ROJA
            ALARM_RGB["r"].on(); ALARM_RGB["g"].off(); ALARM_RGB["b"].off()
        else:
            # Alarma Activada AZUL
            ALARM_RGB["r"].off(); ALARM_RGB["g"].off(); ALARM_RGB["b"].on()
        toggle = not toggle
        _alarm_rgb_stop.wait(0.4)
    alarm_rgb_off()

# ===== LOGICA BTB (HC-5950) =====
```

Para la alarma y los puertos que usa

```

def on_pir_motion_exterior():
    global outdoor_light_on, _last_no_motion
    now = time.time()
    if now < _pir_ready_at:
        return
    if (now - _last_no_motion) < PIR_MIN_NO_MOTION_SEC and not outdoor_light_on:
        return

    with pir_lock:
        if not outdoor_light_on:
            OUTDOOR_LED.on()
            outdoor_light_on = True
            now_iso = utcnow_iso()
            print(f"☀️ Patio: ENCENDIDO por movimiento [{now_iso}]")
            guardar_evento({
                "sensor": "PIR HCSR501",
                "accion": "MOVIMIENTO ON",
                "area": "Patio",
                "fechaHora": now_iso
            }, coleccion=MOV_COLL)
            _cancel_off_timer()

```

Maneja detección de movimiento PIR: enciende luz exterior y guarda evento en MongoDB.

```

def on_pir_no_motion_exterior():
    global _last_no_motion
    _last_no_motion = time.time()
    print("Patio: sin movimiento, iniciando temporizador de apagado...")
    _start_off_delay()

```

Maneja falta de movimiento: inicia temporizador para apagar luz exterior después de delay.

```
# ===== LOGICA DE PORTÓN =====
SERVO_PIN = 12
servo = AngularServo(SERVO_PIN, min_angle=0, max_angle=90,
                     min_pulse_width=0.0005, max_pulse_width=0.0025)

def gate_open():
    try:
        servo.angle = 90
    except Exception as e:
        print(f" Error moviendo servo (abrir): {e}")

def gate_close():
    try:
        servo.angle = 0
    except Exception as e:
        print(f" Error moviendo servo (cerrar): {e}")

def handle_entrance(payload: dict, raw: str):
    action = (payload.get("action") or "").lower()
    now_ts = utcnow_iso()
    if action == "open":
        gate_open()
        print("🚪 Portón: ABRIENDO (90°)")
        lcd_show_message("Abriendo porton", duration_sec=4)
        guardar_evento({
            "evento": "gate_open",
            "fechaHora": now_ts,
            "source": "raspberrypi"
        }, coleccion=VENT_COLL)
```

Para abrir o cerrar el portón que será el servo motor

```
def handle_fan_command(payload: dict, raw: str):
    global fan_manual_on
    action = (payload.get("action") or "").lower()
    now_ts = utcnow_iso()

    if action == "on":
        fan_manual_on = True
        sync_fan_hw()
        print("🌀 Ventilador: ON (manual)")
        lcd_show_message("Ventilador ON", duration_sec=3)
        guardar_evento({
            "topic": TOPIC_FAN, "evento": "vent_on_manual", "device": "ventilador",
            "action": "on", "payload": payload, "fechaHora": now_ts, "source": "raspberrypi"
        }, coleccion=VENT_COLL)

    elif action == "off":
        fan_manual_on = False
        sync_fan_hw()
        print("🌀 Ventilador: OFF (manual)")
        lcd_show_message("Ventilador OFF", duration_sec=3)
        guardar_evento({
            "topic": TOPIC_FAN, "evento": "vent_off_manual", "device": "ventilador",
            "action": "off", "payload": payload, "fechaHora": now_ts, "source": "raspberrypi"
        }, coleccion=VENT_COLL)
```

Controla el ventilador mediante comandos MQTT ON/OFF y guarda eventos en MongoDB.

```
datos: except Exception as e: print(f"Error publicando alerta manual: {e}")
guardar_evento(manual_alert, coleccion=ALERTS_COLL)
```

```
# ===== HELPERS VENTILACION/ALARMA =====
def guardar_evento_par_ventilacion(on_off: str, temp, hum, now_ts):
    guardar_evento({
        "evento": on_off, "sensor": "DHT11", "gpio": DHT_PIN,
        "tempC": round(float(temp), 1),
        "humedad": round(float(hum), 1) if hum is not None else None,
        "fan": "on" if on_off == "vent_on" else "off",
        "fechaHora": now_ts,
        "source": "raspberrypi"
    }, coleccion=VENT_COLL)

def activate_ventilation(temp, hum, now_ts):
    global fan_auto_on
    with alarm_lock:
        if fan_auto_on: return
        fan_auto_on = True
        sync_fan_hw()
        print(f" VENTILACIÓN ON (AUTO): {temp:.1f}°C (>= {VENT_ON}°C)")
        guardar_evento_par_ventilacion("vent_on", temp, hum, now_ts)

def deactivate_ventilation(temp, hum, now_ts):
    global fan_auto_on
    with alarm_lock:
        if not fan_auto_on: return
        fan_auto_on = False
        sync_fan_hw()
        print(f" VENTILACIÓN OFF (AUTO): {temp:.1f}°C (<= {VENT_OFF}°C)")
```

Activan y desactivan ventilación automática basado en umbrales de temperatura.

```
def activate_alarm(temp, hum, now_ts):
    global alarm_auto_on
    with alarm_lock:
        if alarm_auto_on: return
        alarm_auto_on = True
        ensure_alarm_running()
        print(f" 🚨 ALARMA ON (AUTO): {temp:.1f}°C (>= {ALARM_ON}°C)")
        alert_doc = {
            "type": "alarm_overheat",
            "evento": "alarm_on",
            "sensor": "DHT11",
            "gpio": DHT_PIN,
            "tempC": round(float(temp), 1),
            "humedad": round(float(hum), 1) if hum is not None else None,
            "buzzer": "on",
            "rgb_alarm": "red_blue_alt",
            "fechaHora": now_ts,
            "source": "raspberrypi"
        }
        guardar_evento(alert_doc, coleccion=ALERTS_COLL)
        publish_alert(temp, hum, reason="alarm_on")

def deactivate_alarm(temp, hum, now_ts):
    global alarm_auto_on
    with alarm_lock:
        if not alarm_auto_on: return
        alarm_auto_on = False
        # Solo detenemos si además NO está prendida por manual
        if not alarm_manual_on:
```

Activan/desactivan alarma automática por temperatura alta con notificaciones.

```
# ===== LÓGICA ILUMINACIÓN (MQTT) =====
def handle_illumination(payload: dict, raw: str):
    device = (payload.get("device") or "").lower()
    action = (payload.get("action") or "").lower()
    msg_ts = payload.get("timestamp")
    now_ts = utcnow_iso()

    base_doc = {
        "topic": TOPIC_ILUM, "device": device, "action": action, "payload": payload,
        "receivedAt": now_ts, "timestampMsg": msg_ts, "source": "raspberrypi", "status": "executed"
    }

    if device == "led_room":
        room = (payload.get("room") or "").lower()
        led = ROOM_LEDS.get(room)
        if not led:
            print(f" Room desconocido: {room}")
            base_doc.update({"status": "ignored", "reason": f"room_not_found:{room}"})
            guardar_evento(base_doc); return

        if action == "on":
            led.on(); print(f" {room}: ENCENDIDO")
            lcd_show_message(f"{room} ENCENDIDO", duration_sec=4)
        elif action == "off":
            led.off(); print(f" {room}: APAGADO")
            lcd_show_message(f"{room} APAGADO", duration_sec=3)
        else:
            print(f" Acción inválida para led_room: {action}")
            base_doc.update({"status": "ignored", "reason": f"invalid action:{action}"})
```

Procesa comandos MQTT de iluminación: control de habitaciones y RGB con validaciones.

```
# ===== LECTURA DHT11 / GUARDADO =====
DHT_SAVE_EVERY = 300.0 # 5 minutos
def dht_worker():
    try:
        delay_first = DHT_PERIOD_SECONDS - (int(time.time()) % DHT_PERIOD_SECONDS)
        time.sleep(delay_first)
    except Exception:
        pass

    last_saved = 0.0

    while True:
        humedad, temp = Adafruit_DHT.read_retry(DHT_SENSOR, DHT_PIN, retries=3,
        now_ts = utcnow_iso()
        now = time.time()

        if humedad is not None and temp is not None:
            temp = float(temp)
            humedad = float(humedad)

            # LCD: Línea 1 Temperatura y Humedad
            lcd_show_temp(temp, humedad)

            # === Ventilación (AUTOMÁTICO) ===
            if temp >= VENT_ON and not fan_auto_on:
                activate_ventilation(temp, humedad, now_ts)
            elif temp <= VENT_OFF and fan_auto_on:
                deactivate_ventilation(temp, humedad, now_ts)
```

Hilo que lee sensor DHT11 periódicamente, controla ventilación/alarma automática y guarda datos.

```
# ===== LOGICA DE RIEGO =====
def read_state_window():
    n = max(1, int(SAMPLE_WINDOW / SAMPLE_PERIOD))
    dry_count = 0; raw_last = 0
    for _ in range(n):
        raw = 1 if soil_d0.value else 0
        raw_last = raw
        is_dry = (raw == 1) if D0_IS_DRY_HIGH else (raw == 0)
        if is_dry: dry_count += 1
        time.sleep(SAMPLE_PERIOD)
    frac_dry = dry_count / n
    if frac_dry >= DRY_THRESHOLD: return "dry", frac_dry, raw_last
    elif frac_dry <= WET_THRESHOLD: return "wet", frac_dry, raw_last
    else: return "unknown", frac_dry, raw_last

def label_state(state):
    return "SECO" if state == "dry" else ("HÚMEDO" if state == "wet" else

def irrigation_worker():
    bomba_on = False
    last_change = 0.0
    last_log = 0.0
    ignore_until = 0.0
    state_hold = "unknown"
    on_start_time = None
    last_saved = 0.0

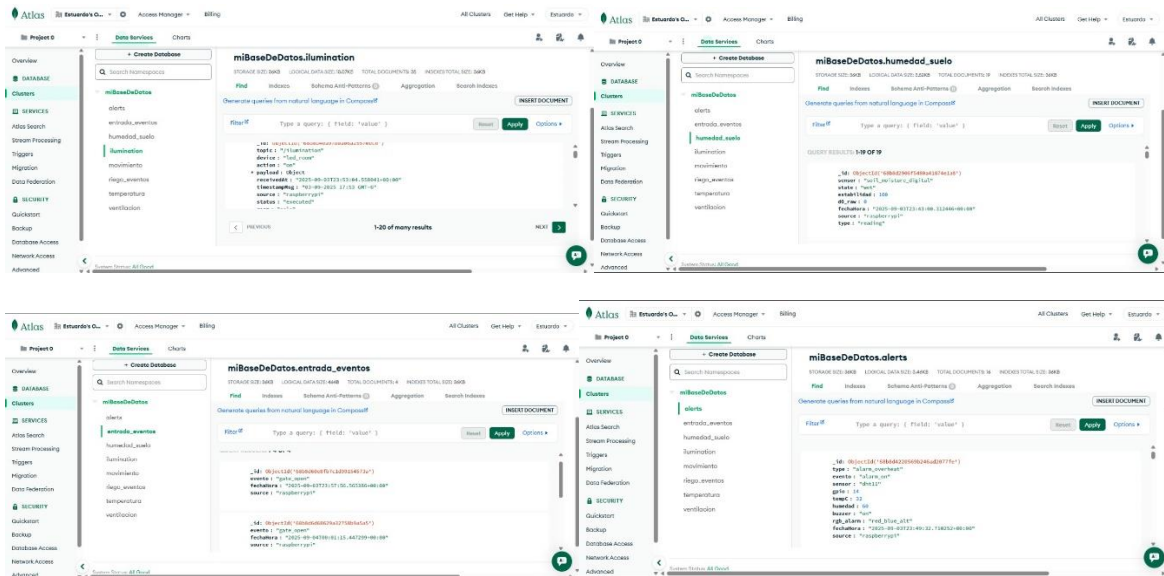
    print(" Riego D0 (integrado) - iniciado")
    try:
```

Monitorean humedad del suelo y controlan bomba de riego automáticamente con lógica de estados.

```
# ===== MQTT CALLBACKS =====
def on_connect(client, userdata, flags, rc, properties=None):
    if rc == 0:
        print(" Conectado a HiveMQ Cloud")
        client.subscribe(TOPIC_ILUM, qos=0)
        client.subscribe(TOPIC_ENTR, qos=0)
        client.subscribe(TOPIC_FAN, qos=0)
        client.subscribe(TOPIC_PUMP, qos=0)
        client.subscribe(TOPIC_ALARM, qos=0)
        print(f" 📻 Escuchando en: {TOPIC_ILUM}, {TOPIC_ENTR}, {TOPIC_FAN}, {TOPIC_PUMP}, {TOPIC_ALARM}")
    else:
        print(" ❌ Error de conexión:", rc)
```

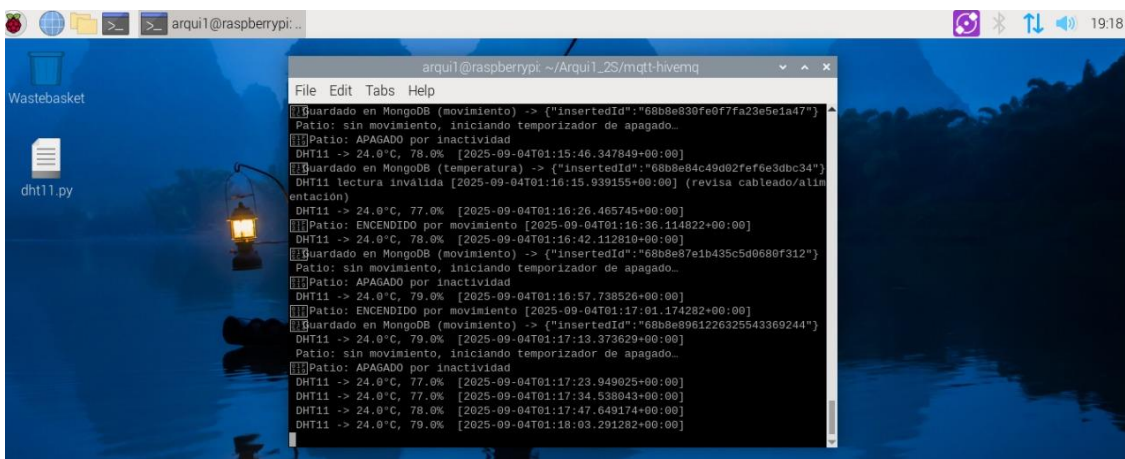
Callbacks de MQTT que manejan conexión al broker y procesamiento de mensajes entrantes.

- Base de datos (MongoDB)



Estas son las tablas y los datos que tenemos en la base de datos en MongoDB, en esta podemos ver los datos de iluminación, humedad del suelo, los eventos de entrada y por ultimo los datos de las alertas podemos notar como es que se reciben y guardan los datos en la base de datos de MongoDB y cual es el formato en el que están organizados los datos.

Raspberry



Es la consola de la raspberry que utilizamos en este caso fue el modelo pi3 con 1gb de RAM

Maqueta y colocación de la electrónica

